Guion del expositor — OWASP Amass

Expositor: Michael Stiven Tabares Tobón
Duración: 25 minutos (8 min teoría + 17 min práctica)
Público: Estudiantes de noveno semestre de Ingeniería de Sistemas (UdeA). El profesor es experto en seguridad.

Antes de empezar: cómo se ejecuta todo

Dato clave para la demo: todo corre dentro de Docker. Nunca escribimos `amass` directamente en el portátil.

Hay una sola carpeta de trabajo y un solo contenedor donde se ejecutan las órdenes:

```
# La carpeta donde vive el laboratorio
cd lab-infra/docker

# Para ejecutar cualquier comando de Amass, se mete al contenedor:
docker compose -f compose.yaml exec amass amass subs -names -d lab.local -ip
```

¿Hay que correr algo de Docker antes? Sí, una sola vez al principio:

```
./start.sh --reset
```

Ese comando construye las imágenes, levanta el laboratorio y comprueba que todo funciona. Tarda unos 2 minutos la primera vez.

Para la demo usaré el atajo que hace todo en un paso:

```
./start.sh --enum
```

> **Nota para el expositor:** si una orden falla, no pasa nada. `./start.sh --results` vuelve a mostrar los resultados sin re-ejecutar nada, y `./start.sh --dns` muestra todas las consultas que se hicieron al DNS.

Diapositiva 1 · Portada

0:00 – 0:30 (30 s)

→ Presentación en pantalla. Saluda, no leas la portada.

> "Buenos días. Soy Michael Stiven Tabares Tobón, estudiante de noveno semestre de Ingeniería de Sistemas. Hoy voy a hablar de OWASP Amass, una herramienta de reconocimiento, y más importante que la herramienta, vamos a construir un laboratorio donde usarla de verdad."

> "El laboratorio corre íntegramente en mi portátil, en contenedores de Docker. Ninguna consulta sale a Internet, y el dominio que vamos a atacar lo creé para esta sesión."

Diapositiva 2 · El problema

0:30 – 1:40 (70 s)

→ Señala las tres cifras.

> "Empecemos por el problema. Casi todas las organizaciones tienen una lista de sus dominios principales. Muy pocas tienen una lista de sus subdominios."

> "Y la diferencia no es trivial. Un subdominio olvidado puede ser un servidor de pruebas, un pipeline de despliegue, una instancia vieja de Jenkins, un panel de administración. Cada uno es una puerta que nadie vigila."

> "El dato de la derecha es el importante: el inventario oficial y la superficie de ataque real son dos conjuntos distintos. Esa brecha es el riesgo."

> "¿Por qué existe? Porque construir el inventario de activos da trabajo, y ese trabajo solo se prioriza después de un incidente. Es reactivo."

> "OWASP Amass responde una pregunta concreta: ¿qué activos existen realmente y cuáles importan?"

Diapositiva 3 · ¿Qué es OWASP Amass?

1:40 – 2:50 (70 s)

→ Lee la definición, luego las tres tarjetas.

> "OWASP Amass es un framework open source de la organización OWASP para reconocimiento de activos y mapeo de superficie de ataque."

> "Tres cosas lo diferencian de las herramientas que ustedes ya conocen."

> "Uno: no es solo un enumerador de subdominios. Un subdominio aislado es ruido. Amass lo conecta con su IP, con el servicio que corre ahí, con el certificado TLS y con las fuentes donde lo encontró."

> "Dos: construye un grafo. Cada activo es una entidad, cada relación es una arista. El resultado es un mapa, no una lista."

> "Tres: persiste y deja trazabilidad. Cada hallazgo guarda de dónde salió. Eso permite comparar la superficie de ataque entre meses."

> "La diferencia con `dig` es de fondo: `dig` responde una pregunta. Amass construye el mapa de lo que esa respuesta implica."

Diapositiva 4 · Pasivo vs. Activo

2:50 – 4:30 (100 s)

→ Columna PASIVO, luego ACTIVO.

> "Esta es la decisión más importante de la práctica. Les pido que la retengan."

> "Modo pasivo: Amass no toca el objetivo. Consulta fuentes públicas: Certificate Transparency, historial de DNS, motores de búsqueda. Lo que detecta es lo que otros ya publicaron. El tráfico hacia el objetivo es cero."

> "Modo activo: Amass interroga la infraestructura. Resuelve nombres y comprueba servicios. Detecta lo que existe ahora, incluidos hosts que no están en ningún índice público."

> "En una auditoría real se usan los dos. Pero lo importante es entender qué se está tocando."

> "Y quiero ser claro, porque el profesor seguro va a preguntarlo: esto que vamos a hacer hoy contra un DNS real sin autorización no es unethical. Es un delito. La línea entre investigación y ataque no está en la herramienta. Está en el permiso."

Diapositiva 5 · Arquitectura

4:30 – 5:30 (60 s)

→ Recorre el pipeline con el dedo.

> "Por dentro es un pipeline de cuatro etapas."

> "Uno: las fuentes. Sitios donde puede haber información: fuentes OSINT, el DNS autoritativo, listas de palabras."

> "Dos: el motor. Son plugins especializados. Uno consulta Certificate Transparency, otro resuelve DNS, otro busca certificados. El motor los coordina."

> "Tres: la base de datos. Aquí está la diferencia con un script de Bash. Cada activo se guarda como entidad tipada y cada relación como una relación entre ellas."

> "Cuatro: la salida. El subcomando `subs` para leer resultados, `track` para comparar enumeraciones, `viz` para visualizar, y exportación a JSON."

> "Abajo está el ejemplo: un nombre, su IP, el servicio que corre en esa IP y la fuente con su nivel de confianza. Eso es lo que queda guardado. Por eso el resultado es auditable."

Diapositiva 6 · La base de datos

5:30 – 6:40 (70 s)

→ Señala las tres columnas.

> "Vale la pena detenerse en la base de datos, porque alguien seguro va a preguntar y la respuesta es más interesante de lo que parece."

> "Sirve para tres cosas."

> "**Persistir.** Cada ejecución queda guardada. Eso permite comparar la superficie de hoy contra la de la semana pasada y ver qué cambió: un subdominio nuevo, un puerto nuevo, una IP que se movió."

> "**Correlacionar.** Si un activo lo encontraron tres fuentes distintas, se guarda una sola vez anotando las tres. Se elimina el ruido."

> "**Priorizar.** Guarda el TTL, la confianza y el origen de cada dato. Permite distinguir un hallazgo sólido de una simple pista."

> "En el laboratorio usamos SQLite porque se crea solo. PostgreSQL es lo que usaría un equipo con histórico compartido."

Diapositiva 7 · Divisor "Manos a la obra"

6:40 – 8:00 (80 s)

→ Transición. Aquí cambias de registro: de expositor a operador.

> "Hasta aquí la teoría. Ahora vamos a ejecutar."

> "Y antes de la primera orden, déjenme mostrarles cómo está armado el laboratorio, porque de ahí se entiende todo lo que vamos a ver."

→ Señala el panel de topología.

> "Son dos piezas. Una hace de servidor DNS y sirve una zona ficticia con 32 nombres. La otra tiene Amass y hace el reconocimiento. Las dos en una red privada, `192.168.56.0/24`."

> "Todo esto corre en contenedores de Docker, así que cualquiera de ustedes puede reproducirlo en su portátil con un solo comando. Eso lo aclaro porque varios me han preguntado si necesitan Kali."

> "Un detalle que descubrí leyendo el código: en la versión 5 de Amass, el módulo de DNS no usa el servidor DNS que uno le configure en el archivo de configuración. Usa una lista de servidores públicos que están escritos directamente en el programa. Para que las consultas llegaran a nuestro DNS ficticio, hubo que hacer una redirección a nivel de red."

> "No voy a entrar en detalles de redes, pero quiero que lo sepan: las consultas DNS que van a ver en los logs **sí** salen del contenedor, y llegan a nuestro servidor. Eso es lo que hace posible todo lo que viene."

Diapositiva 8 · Paso 0 — Verificar el entorno

8:00 – 10:00 (120 s)

→ Abre una terminal en la carpeta del proyecto.

> "Paso cero. Antes de ejecutar Amass, comprobamos que el laboratorio está en pie."

Orden 1 — el DNS del laboratorio responde:

```
cd lab-infra/docker
docker compose -f compose.yaml exec amass \
  dig +short @192.168.56.101 dev.lab.local A
```

> "Aquí está: `dev.lab.local` resuelve a 192.168.56.103. El DNS ficticio está sirviendo la zona."

Orden 2 — un nombre que no existe:

```
docker compose -f compose.yaml exec amass \
  dig +short @192.168.56.101 test.lab.local A
```

> "Aquí no hay respuesta. `test` no existe en la zona. Esto lo van a ver de nuevo en la fuerza bruta: el DNS es el que decide qué existe."

Orden 3 — versión de Amass:

```
docker compose -f compose.yaml exec amass amass -version
```

> "Amass 5.1.3. Y quiero detenerme aquí, porque es una diferencia con lo que van a encontrar en tutoriales viejos."

> "En la versión 5, el subcomando `enum` ya no tiene las opciones `-src`, `-o` ni `-ip`. Y el subcomando `db` fue eliminado. Los resultados se leen con `subs`, que consulta la base de datos. Es un cambio que ha roto muchos scripts."

> "Perfecto. El laboratorio está listo. Vamos a usarlo."

Diapositiva 9 · Paso 1 y 2 — Enumeración y fuerza bruta

10:00 – 13:00 (180 s)

→ Ejecuta la enumeración.

> "Paso uno: enumeración activa con fuerza bruta."

```
cd lab-infra/docker
docker compose -f compose.yaml exec amass amass enum \
  -engine http://127.0.0.1:4000 \
  -active \
  -brute -w /wl/subdomains-lab.txt \
  -d lab.local \
  -p 80,443 \
  -timeout 3
```

> "Déjenme explicar cada parte antes de que termine."

> " `-active` : que pregunte al DNS y compruebe los puertos."

> " `-d lab.local` : el objetivo."

> " `-p 80,443` : solo esos dos puertos. Es el muro de seguridad del ejercicio."

> " `-brute -w` : la fuerza bruta. En vez de adivinar nombres, Amass **pregunta al DNS** si cada candidato existe. La lista tiene 13 palabras."

> " `-timeout 3` : si deja de avanzar tres minutos, se detiene solo."

→ Muestra la barra de progreso.

> "Veintinueve de treinta. El motor hizo más de 500 consultas al DNS."

Orden para leer los resultados:

```
docker compose -f compose.yaml exec amass amass subs -names -d lab.local -ip
```

> "Y aquí están, con su IP correlacionada."

→ Señala la salida.

> "Ahora lo importante: fíjense en lo que **no** aparece."

> "En la lista hay 13 candidatos. Once existen en la zona y dos no. Los dos que no existen fueron descartados. Y no los descartó Amass: los descartó el DNS."

> "Y eso es el punto pedagógico de la práctica: **la fuerza bruta no solo confirma, también descarta**. Un escáner que solo te dice lo que encontró, sin decirte lo que rechazó, no te deja saber si tu escaneo fue completo."

Evidencia en el DNS (si hay tiempo):

```
cd lab-infra/docker
docker compose -f compose.yaml logs dns | grep query | \
  sed 's/.*\[query\] //' | awk '{print $1}' | sort -u | grep -E "\.lab\.local$"
```

> "Aquí está la bitácora completa de lo que se preguntó. `test.lab.local` aparece ahí: se preguntó y el DNS dijo que no."

Diapositiva 10 · Paso 3 — Del hallazgo al riesgo

13:00 – 16:00 (180 s)

→ Señala la tabla de severidad.

> "Ahora la parte que separa una herramienta de un análisis."

> "Pasamos de tener nombres a tener un inventario priorizado. Y la priorización no es automática."

> " `vpn.lab.local` es crítico: un solo activo da acceso remoto a toda la organización."

> " `dev` , `staging` y `qa` son altos: son preproducción, con menos controles que producción y a veces con datos reales."

> " `jenkins` y `gitlab` son altos por otra razón: son la cadena de suministro. Acceso al código y a los pipelines es acceso a todo lo que se despliega después."

> " `old.lab.local` es medio: activo residual, deuda técnica sin actualizar."

> "Y `www` y `mail` son bajos: activos públicos esperados."

→ Señala el recuadro inferior.

> "Pero lean el recuadro con atención, porque es el mensaje final."

> "**Amass no prioriza.** Entrega el inventario con su procedencia. La severidad depende del contexto: si esa IP es producción o una máquina de pruebas, el mismo hallazgo cambia por completo. Eso lo decide el equipo con conocimiento del negocio, no el script."

> "Automatizar la recolección es fácil. Interpretar el resultado es el trabajo de verdad."

Diapositiva 11 · Cierre

16:00 – 22:00

(Si la demo fue bien, este bloque puede ser más corto y dejar más tiempo a preguntas.)

→ Señala las tres tarjetas.

> "Tres ideas para llevarse."

> "**Primero: Amass es un grafo, no una lista.** El valor está en las relaciones, no en el dato aislado."

> "**Segundo: pasivo y activo responden preguntas distintas.** Ambos son legítimos, pero exigen autorización y alcance distintos. Elegir bien es lo que separa un mapeo de superficie de ataque de un escaneo indiscriminado."

> "**Tercero: el límite real es el alcance ethical.** La herramienta se automatiza; la responsabilidad no. Entender el alcance es más difícil que dominar los comandos, y es lo que nos va a diferenciar como profesionales."

> "Y antes de cerrar, que quede claro: todo lo que vieron hoy corrió en contenedores en mi portátil. Ni una sola consulta salió a Internet. El laboratorio es real en la mecánica y seguro en el alcance."

> "El código, la zona DNS y los scripts están en GitHub. Con gusto los revisamos."

→ Pausa. Preguntas.

Plan B — Si algo falla

Problema	Qué decir / hacer
El comando tarda o se cuelga	`Ctrl+C` y sigue. Di: "esto pasa cuando el aula va lenta; el resultado ya está guardado". Luego `./start.sh --results`.
El DNS no responde	`./start.sh --check` para ver el estado. Si el contenedor DNS está caído: `docker compose -f compose.yaml up -d dns`
Un comando da error	Di: "esto es una de las diferencias de la versión 5 que mencioné".

	Pasa a la siguiente orden. Un error explicado suma credibilidad.
No hay Internet en el aula	No es problema: **el laboratorio no necesita Internet**. Enfatízalo: "por eso podemos hacer esto en un avión".
Faltó tiempo	Sáltate el paso de `--dns` y ve directo a la diapositiva 10.

Comandos de la demo — resumen en una tabla

#	Para qué	Comando
1	Levantar el laboratorio	`./start.sh --reset`
2	Verificar el DNS	`docker compose -f compose.yaml exec amass dig +short @192.168.56.101 dev.lab.local A`
3	Versión de Amass	`docker compose -f compose.yaml exec amass amass -version`
4	Enumeración + fuerza bruta	`docker compose -f compose.yaml exec amass amass enum -engine http://127.0.0.1:4000 -active -brute -w /wl/subdomains-lab.txt -d lab.local -p 80,443 -timeout 3`
5	Ver resultados	`docker compose -f compose.yaml exec amass amass subs -names -d lab.local -ip`
6	Ver consultas DNS	`./start.sh --dns`
7	Resultados sin re-ejecutar	`./start.sh --results`
8	Detener	`./start.sh --stop`

Todos los comandos con `docker compose` se ejecutan desde `lab-infra/docker`.

Control de tiempo

Tramo	Diapositivas	Minutos
Apertura	1	0.5
El problema	2	1.2
Qué es Amass	3	1.2
Pasivo vs. Activo	4	1.7
Arquitectura	5	1.0
Base de datos	6	1.2
Transición	7	1.2
Total teoría	**1–7**	**8.0**
Verificación	8	2.0
Enumeración + fuerza bruta	9	3.0
Interpretación	10	3.0
Cierre	11	2.5
Preguntas	—	3.5
Total		**25.0**

Notas para el expositor

- **No leas las diapositivas.** Están para el público; tú cuentas la historia.

- **La demo es el corazón.** Si entrenas una cosa, entrena la demo.
- **Si algo falla, explícalo.** Un error técnico explicado suma credibilidad; un resultado sin explicar la destruye.
- **El profesor es experto.** Si sale una pregunta técnica muy específica, no improvises: di "eso lo vi en el código fuente" y sigue. Es honesto y demuestra rigor.
- **Cierra con la frase de la diapositiva 11:** "La herramienta se automatiza; la responsabilidad no."
- **Decide ahora qué hacer con Metasploitable 3:** no lo montamos, y en la diapositiva 7 lo omitimos. Si en el momento alguien pregunta "¿y no/Xavier had an attack surface?", la respuesta es: "el objetivo real de la demo es que Amass no sepa qué hay; en un laboratorio con objetivo propio eso no aplica".